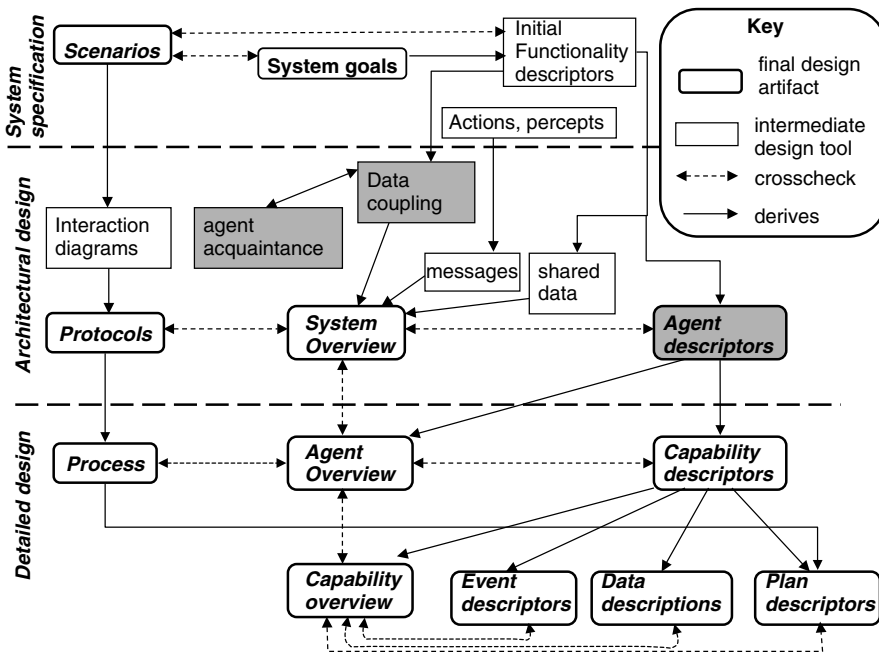


Architectural Design: Specifying the Agent Types



Phases, artifacts and relationships in the design process

In the system specification phase we developed system goals, scenarios and functionality descriptions. In the next three chapters we will cover the *architectural design* phase where we use these artifacts as the basis for developing the high-level design of the agent system. As noted earlier, it is expected that all the phases interact with each other,

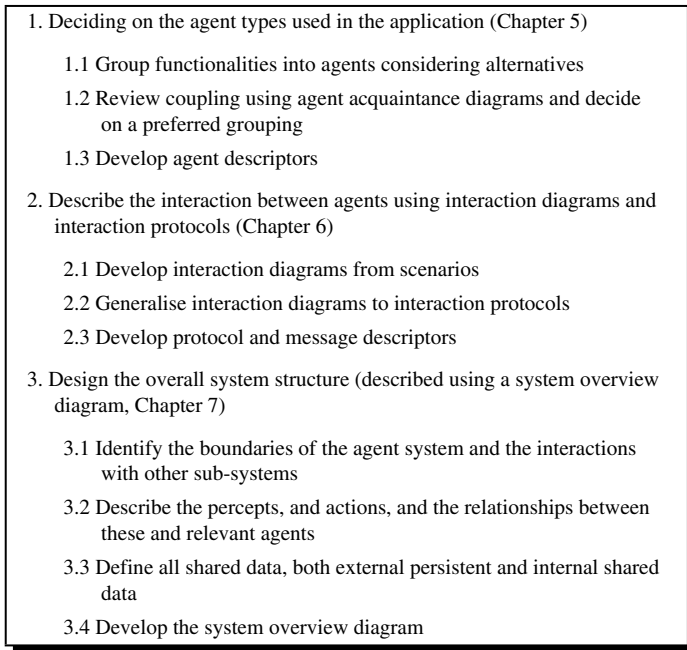


Figure 5.1 The process of architectural design

and that developing the design is an iterative process, also involving implementation. The aspects of architectural design covered in this and the subsequent two chapters interact with each other as well as with system specification.

The three aspects that are developed during architectural design, as shown in Figure 5.1 are:

1. Deciding on the *agent types* used in the application.
2. Describing the interactions between agents using *interaction diagrams* and *interaction protocols*.
3. Designing the overall system structure (described using a *system overview diagram*).

This chapter focuses primarily on the process of deciding on the agent types to be used, while Chapter 6 focuses on defining the interactions, and Chapter 7 covers the system overview.

The architectural design defines what agents are to be a part of the system and how these agents interact with each other to meet the required functionality of the system. It defines the interface of the system to the external environment, and also the interfaces of the individual agents within the system. As the design develops, it is also likely that aspects of the previous stage are revisited and revised, to achieve a cleaner architectural design.

A major decision that is made during the architectural design is the types of agents used. Agent types are formed by combining functionalities. The choice of how functionalities are to be combined is made by considering the functionalities and scenarios and

developing possible groupings of functionalities into agents, which are then evaluated according to the standard software engineering criteria of *coupling* and *cohesion*. One proposed grouping of functionalities is better than another if it gives lower coupling and higher cohesion.

☞ DEFINITION: *Coupling and Cohesion*

Coupling: Coupling is a property of a group of components. The coupling of components is the extent to which they depend on one another. If a component A depends on component B, then changing B may require changing A as well. Low coupling yields a system in which changes are more likely to be localized. High coupling yields a system in which changing one component is likely to require changes to many other components, thus making the system harder to change. *In Agent systems coupling is exhibited primarily in communication between agents, although use of a shared data store is another possible form of coupling.*

Cohesion: Cohesion is a property of a single component. A component is cohesive if all of its parts are related. For example, an object is cohesive if all its data and methods are related to a single clear goal or functionality. *There are a number of ways in which an agent can exhibit cohesion. For example, it could manage a series of actions that occur at the same time (e.g. initialization). An agent could also be responsible for the operations involving a particular data store, including consistency management. Most often, cohesion in an agent is based on the goals of the agent being closely related.*

As with any design, there is a large space of possibilities and no clear right or wrong decisions. What is provided here are some methods and tools to develop and document a design, as well as some ways to evaluate the comparative merits of alternative design options. The *data coupling diagram* and the *agent acquaintance diagram* are tools used to develop and assess proposed groupings.

Once we have decided upon the agents in the system, we identify the pathways of communication (which agents talk to which other agents) as well as the *timing* of communication (which messages are followed by which other messages). The timing of communication is captured initially in the scenarios: whenever an activity by an agent is followed by an activity of another agent, there is, implicitly, communication between them. This communication is depicted explicitly in agent *interaction diagrams*. Like scenarios, interaction diagrams depict one possible sequence of messages between agents. In order to describe all possible interactions, we develop interaction protocols; depicted using the Agent UML (AUML) notation (Huget *et al.* 2003).

Finally, we specify the interactions between the agents, and the system interface in terms of percepts, actions and external data. We also identify any shared data. The overall design of the system is then depicted in the *system overview diagram*, which brings all the items together. The system overview diagram is perhaps the single most important

product of the design process. It ties together agents, data, external input and output and shows the communication between agents.

 **TIP:** It is possible to show some timing information in a system overview diagram by marking the arcs between agents and messages with numbers. However, our experience has been that, except for very simple cases, this quickly becomes cumbersome and complex. Generally, it is simpler to keep dynamic and static information separate.

5.1 DECIDING ON THE AGENT TYPES

The single most important decision in the architectural design is which agents the system will include. Different options are evaluated using the criteria of *coupling* and *cohesion*. Each agent should be cohesive, in that it can be sensibly described with a short title. The system of agents should be as loosely coupled as possible. It is preferable to have a system in which each agent needs to know about only a limited number of other agents within the system.

The steps we go through in this part of the process are:

1. Group functionalities into agents considering alternative groupings.
2. Review coupling (using agent acquaintance diagrams) and decide on a preferred grouping.
3. Develop agent descriptors.

5.2 GROUPING FUNCTIONALITIES

Given a collection of functionalities, there are obviously many ways in which they can be grouped into agents.¹ At one extreme, every functionality could correspond to a different agent. This is undesirable in that having closely related functionalities in different agents leads to a high degree of dependency, or coupling, between agents. At the other extreme, all of the functionalities could be grouped into a single agent. This is undesirable in that grouping unrelated functionalities together leads to overly complex agents lacking in cohesion.

¹The term ‘agent’ is somewhat overloaded in that it can refer to both the ‘agent type’ and to the ‘agent instance’. The agent type is a template for agents (similar to a class in object-oriented programming), whereas the agent instance is a run-time entity (similar to an object in object-oriented programming). Context determines which use of agent is intended. At times, we explicitly use ‘agent type’ or ‘agent instance’ for extra clarity, but this is cumbersome if used constantly.

The main reasons for deciding to group certain functionalities together into a single agent are:

- The functionalities seem related – it ‘makes sense’ to group them. For example, the *Price setting and Competition management* functionalities in our case study are clearly related.
- The functionalities require a lot of the same information. If grouped into a single agent, this can then be represented in internal agent data structures. If separate agents are used, the information must be passed via messages unless a shared data store is used, which is not usually a good design decision (see Section 7.4 on page 88).

Reasons for *not* grouping functionalities are primarily the following:

- The functionalities are clearly unrelated
- The functionalities exist on different hardware platforms
- Different numbers of functionalities are required at run time.
For example, you might have two related functionalities: an *Online Interaction* functionality, and a *Customer Contact* functionality, both of which have to do with customer interaction. However, the *Online Interaction* functionality is required to be part of an agent type where there is one agent per active user. The *Customer Contact* functionality, on the other hand, is required to be in an agent type where there is a single agent of this type in the system. Although the two functionalities are related, we cannot group them together in a single agent type because we cannot simultaneously have exactly one agent of a given agent type and multiple agents of a given type. We term this situation a *cardinality mismatch*.

Other reasons for not grouping functionalities include *security* and *privacy* – if data associated with one functionality should not be available to another functionality; and *modifiability* – if a functionality will change, or will be modified by different people.

One technique that we use to systematically examine the properties that lead to coupling and cohesion is the *data coupling diagram*. Potential groupings are then evaluated and possibly refined using an *agent acquaintance diagram*. We explain each of these in detail in the following sections.

DATA COUPLING DIAGRAMS

One strong reason for grouping functionalities together is data coupling – they use the same data. In order to more easily visualize groupings suggested by data coupling, we can use the functionality descriptors to develop what we call *data coupling diagrams*.

A data coupling diagram consists of the functionalities and all identified data (not only persistent data but also data that the functionalities require to fulfil their job). Directed links are then inserted between functionalities and data, where an arrow pointing towards the data indicates the data is *produced* or *written* by that functionality, whereas an arrow pointing towards the functionality indicates the data is *used* by the functionality. A double-headed arrow indicates that the functionality both *uses* and *produces* the data.

The diagram must be checked to ensure that all data is produced somewhere, unless it has been determined that it is provided externally on system start-up and is static.

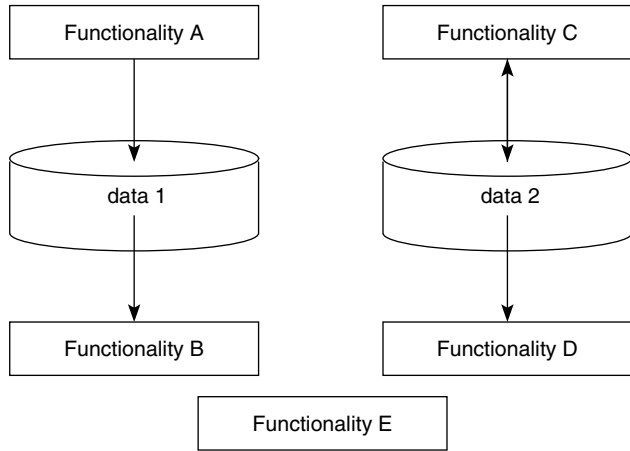


Figure 5.2 Simple data coupling diagram

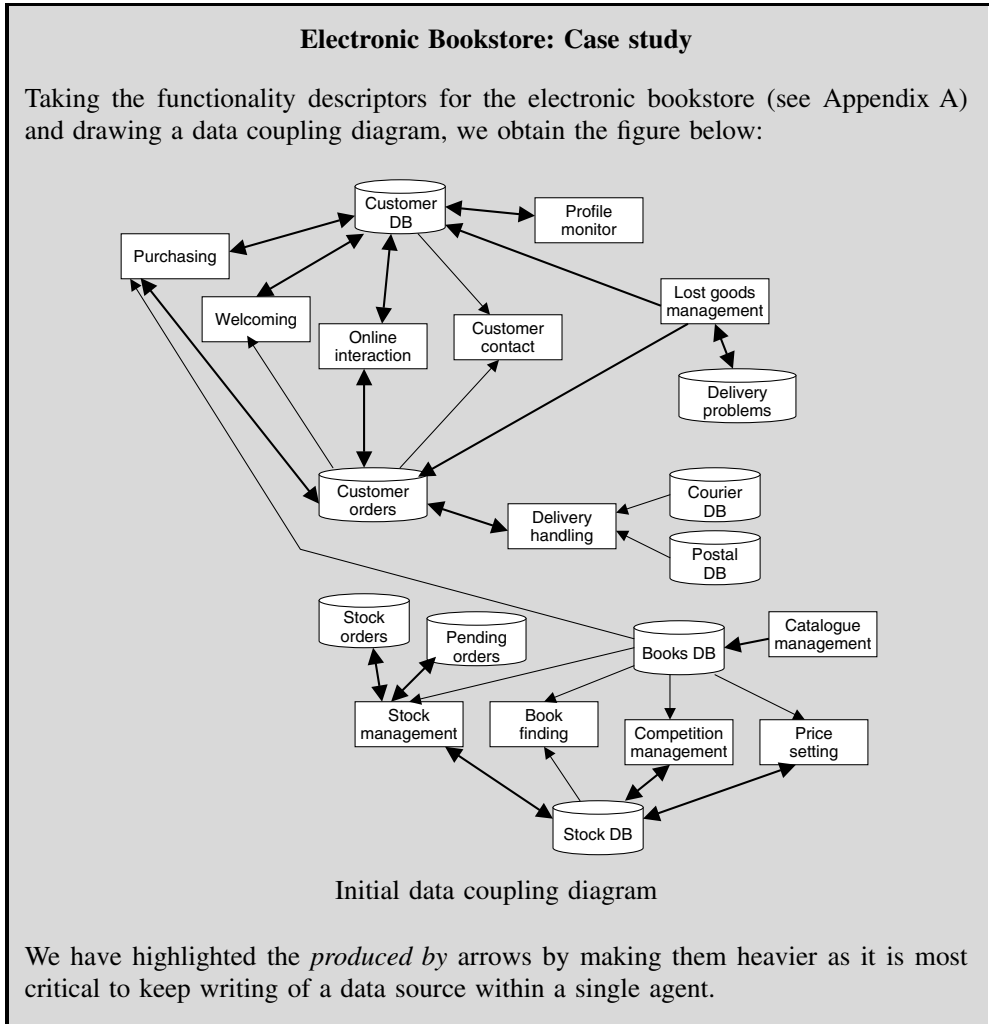
Also, the existence of data that is produced but not used usually indicates an omission elsewhere in the design.

Once the diagram is complete, it can then be assessed visually for groupings that are linked by their data use. When assessing the diagram visually, we are looking for clusters of functionalities around data. Each such grouping must also be assessed for cohesion – do the functionalities fit naturally together into a single area of responsibility? It must also be assessed with respect to whether there is some reason to keep the functionalities separated.

📌 **TIP:** Checking that all data is produced/written by some functionality can often reveal missed ‘data produced’ fields in the functionality descriptors. This is where a functionality should produce certain data, but this fact has not been recorded in the design. This is a typical example of refinements and revisions that are commonly made to the system specification documents during the architectural design stage.

Consider the simple example data coupling diagram in Figure 5.2. We see that A and B exhibit data coupling as do C and D, while E is not data-coupled to anything. Let us assume that A, B, C and E are somewhat conceptually related, while C is also conceptually related to D. Let us also assume that C should not be in the same agent as A, as they need to be on different platforms. We tentatively put A and B into agent X, because of their data coupling, and C and D into agent Y. These decisions are consistent with cohesiveness, and also avoid the undesired grouping of C with A. Since E is conceptually related to A and B, it could be included into agent X without destroying cohesion. Alternatively, we could create a third agent, Z, containing functionality E. Consequently, we decide to consider two alternative groupings: {Agent X = A, B, E; Agent Y = C, D} or {Agent X = A, B; Agent Y = C, D; Agent Z = E}. Both these designs can then be considered further.

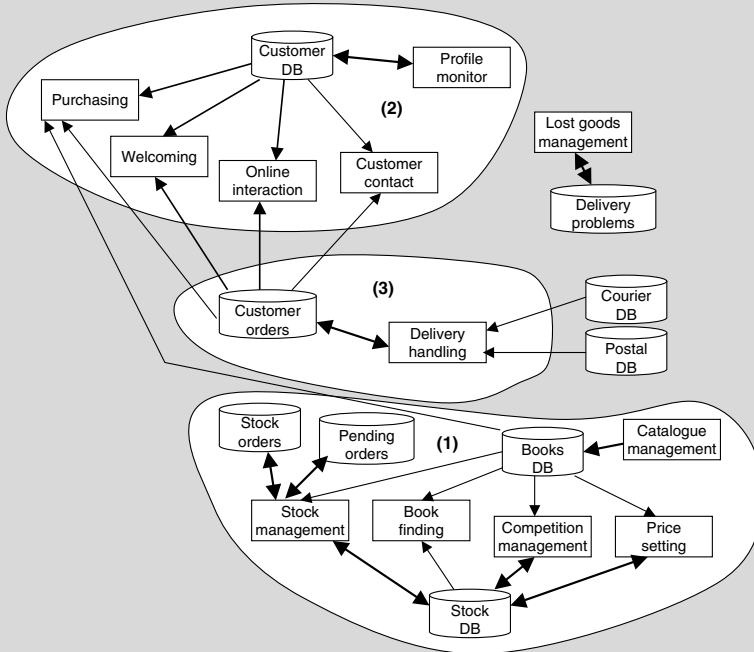
We note that the data coupling diagram does not capture all interaction between functionalities: some interaction may be based on message passing, possibly for transfer of control as well as of data. For instance, a sales transaction functionality may pass control to a delivery management functionality once it has completed its job. We have experimented using diagrams that show this kind of interaction, but have not found these as useful as the data coupling diagrams.



Sometimes, as in the bookstore example, there are initially no clear groupings that emerge from the data coupling diagram. In such a case, it is necessary to then make some design decisions that enable groupings to be made. These may include such things as deciding that a particular functionality will manage all interactions (or at least all writing) with a particular data store, or rethinking the details of scenarios and functionalities to eliminate some of the complexity of data interactions.

Electronic Bookstore: Case study

In the initial bookstore data coupling diagram (shown previously), we observe that there are many *produced by* arrows from different functionalities for the data sources *Customer DB* and *Customer Orders*. We make the design decision that all writes to these data sources will go via the functionalities *Profile Monitor* and *Delivery Handling*. We remove all *produced by* arrows to these data sources, other than from these functionalities. At this point, there emerge three clusters of functionalities (i.e. potential agents) as shown in the following figure.



Natural functionality groupings after removing write links

- The first cluster (1) involves *Stock management* and *Price setting*. The *Stock DB* is written to by both *Price setting* and *Stock management* and so we want to keep the two functionalities in the same agent. The book finding functionality reads both the data stores here and so we (for now) put it in the same agent.
- The second cluster (2) revolves around the *Customer DB* and interactions with the customer. These functionalities read and write the *Customer DB* and some also read *Customer Orders*.
- The third cluster (3) is the functionality and data stores associated with delivering orders. Note that we have moved the data stores

Courier DB and *Postal DB* outside the cluster as we have previously determined that these will be external data stores.

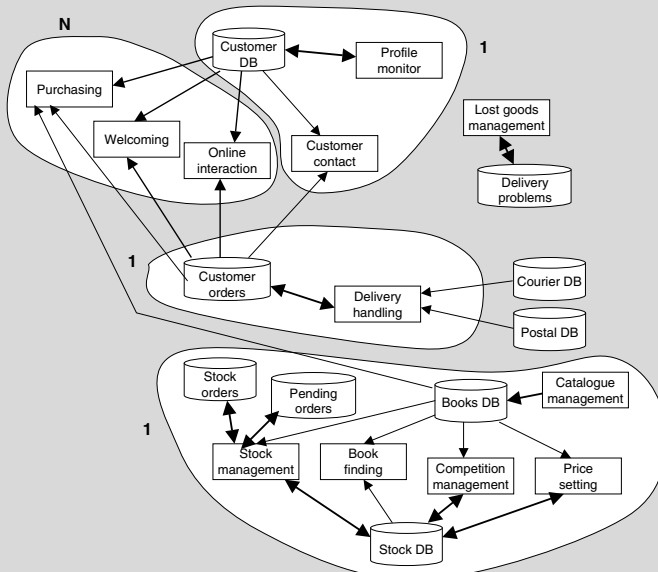
- The functionality for *Lost goods management* stands alone at this stage and could be incorporated into one of the three clusters.

We have indicated previously a number of reasons for and against grouping particular functionalities together, and these must all be considered in relation to the initial groupings suggested by the data coupling diagram. Depending on the situation, these considerations may even result in design decisions being made prior to the initial data coupling diagram. For example, if it is known that certain functionalities are required to reside on a separate hardware platform, a decision may be made early on as to which data stores are required on that platform. Two data coupling diagrams may then be developed, one for each platform, to facilitate further decisions about groupings into agents.

Cardinality is one of the key issues to consider, and this is addressed for the bookstore example that follows.

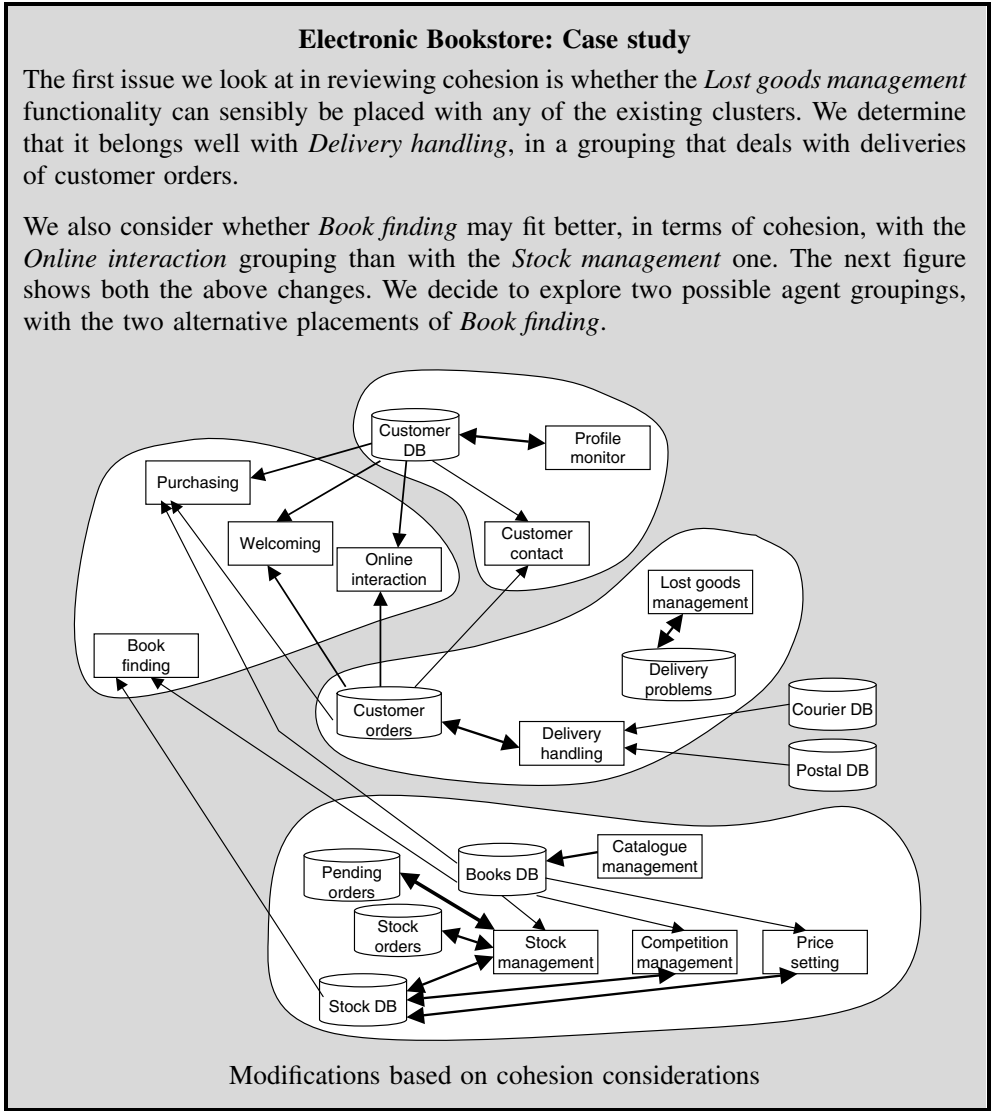
Electronic Bookstore: Case study

In analysing the grouping so far with respect to cardinality issues, we note that the functionalities *Online interaction*, *Welcoming* and *Purchasing* all require one instance per customer, whereas *Profile monitor* and *Customer contact* require one instance for the whole system. Consequently, we split these into two separate groupings as follows. Note that when splitting the clusters, data is kept with the functionalities that produce it.



Revision due to cardinality differences

As has been indicated previously, *cohesion* is extremely important, and whatever clusterings are being considered, cohesion of the resulting groupings is a major concern. Adjustments to groupings may well need to be made or considered in order to improve this design aspect.



The result of the analysis described (step 1 on page 56) is some number of designs where a design is a set of possible agent types and an agent is a collection of functionalities. The next step is to assess each proposed design (i.e. set of agent types) and decide on the most appropriate one.

 **TIP:** A simple but well-used and effective heuristic for assessing cohesiveness is whether a suitable name for the agent can be found that adequately encompasses all of its functionalities. An agent that is cohesive should be able to be described by a single term without any conjunctions ('and'). For example, *Sales Assistant* agent is a simple descriptive name suitable for the combination of the functionalities of *Book Finding*, *Purchasing*, *Welcoming* and *Online interaction*. This suggests that the grouping of functionalities is cohesive. While only a 'rule of thumb', this is often an effective (and quick) way of assessing cohesion in proposed designs.

5.3 REVIEW AGENT COUPLING – ACQUAINTANCE DIAGRAMS

One of our design criteria is to aim for a system that is as loosely coupled as possible. We do not want all agents to have to know about all other agents.

In order to evaluate a potential grouping for coupling, we use an *agent acquaintance diagram*. This diagram represents each of the agent types. Information about agent interaction is extracted from the functionality descriptors and each agent is linked with the other agents it interacts with. Links can be decorated with the cardinality of the relationship if desired (e.g. one *Stock Manager* agent interacts with many *Sales Assistant* agents).

We then analyse the resulting diagram in two ways. One is simply an analysis of the density of the links within the diagram. It is a measure of the ratio of the actual coupling to the maximal possible coupling. If the system has four agents, then each agent could potentially be linked to a maximum of three other agents, giving a total number of $4 \times 3 / 2$ possible links.² To get the link density, we simply count the links and divide by this number.

It is likely that all agents in the system are linked in some way, and if this is the case, the minimum number of links will be one less than the number of agents. An obvious design that has such a number of links is a star design, where each agent is coupled to one central agent. Although this has a low level of coupling, it is an undesirable design for two reasons: firstly, change to the central agent type is likely to affect many of the other agents; and secondly, it is a situation that can potentially lead to a bottleneck at run time when many agents are trying to communicate with a single agent.

Consequently, we also consider bottlenecks. If we ignore agent cardinality, the bottleneck factor can be assessed simply by examining the number of links that an agent has. The largest number of links from an agent type can be used as an indication of the worst bottleneck in the system for the purpose of comparing designs.

²Links are not directional, so a link from A to B is the same as a link from B to A, hence the division by two.

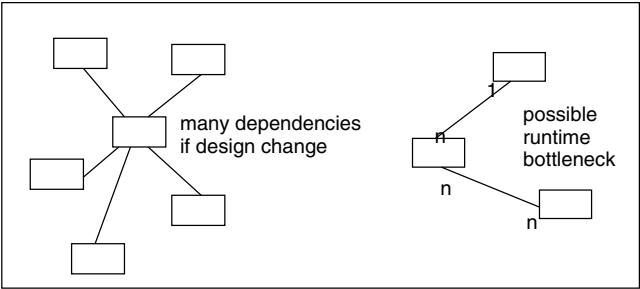
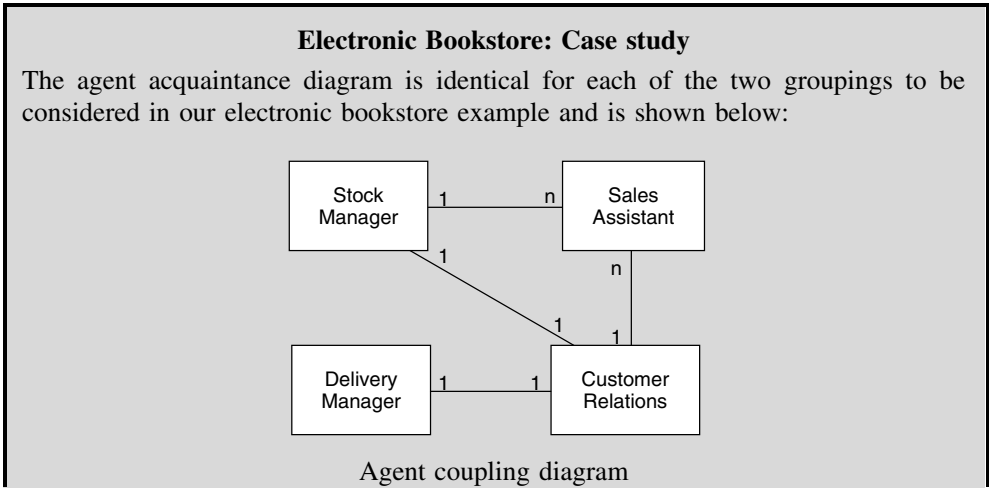


Figure 5.3 It is important to consider different kinds of bottlenecks in the design

However, it is also important to consider situations in which there are some larger numbers of agent instances of a given type as this can produce run-time bottlenecks (Figure 5.3). For example, if we have multiple *Sales Assistant* agent (one per customer) communicating with a single *Stock Manager* agent, this is a potential run-time bottleneck. To decide whether it is actually a problem requires consideration of the actual situation – how many customers do we expect to be dealing with concurrently, and how many interactions do we expect the *Sales Assistant* agent to have with the *Stock Manager* agent? It is not possible to provide a metric for assessing bottlenecks, partly as the number of instances is often not fixed, but rather is a function of system usage – for example, one *Sales Assistant* agent for each customer. Also, the numbers themselves do not necessarily provide useful information. Rather, it is important to consider the possible bottleneck issues and weigh these against the various other factors in deciding the preferred design.

A design with lower link density is less highly coupled and is therefore preferable, all else being equal. However, issues such as bottlenecks and cohesion, as well as possible additional factors such as agent size, must all be considered.



We note that this configuration is quite highly coupled, with a link density of $\frac{4}{6}$. We also observe that there are potential run-time bottlenecks with multiple *Sales Assistant* agents talking to one of the other agents. However, on consideration it is not felt that the number of *Sales Assistant* agents or the volume of communication will be a problem.

Possibly, further design work could lead to a more decoupled design that was also cohesive and met all constraints. However, we decide to continue with the design that incorporates *Book finding* into the *Sales Assistant* agent.

 **TIP:** The agent acquaintance diagram can also be used to review whether all the links are in fact necessary. Often, in reviewing an agent acquaintance diagram some links seem natural – you would expect those agents to interact – whereas others appear less intuitive. It is often possible to eliminate some of the less intuitive interactions by revisiting the scenarios and revising ideas as to what should happen when. This can often lead to a cleaner, more decoupled design than was initially possible.

5.4 DEVELOP AGENT DESCRIPTORS

The decision as to which design is to be pursued is made after reviewing the agent acquaintance diagrams and considering issues such as agent size and cohesion.

During and after making the decision as to which agent types we will have within the system, there are a number of questions that need to be resolved:

- How many agents of each type will there be (singleton, a set number, or multiple instances based on dynamics of the system, for example one *Sales Assistant* agent per customer that arrives at a website)?
- What is the lifetime of each agent? If agents are created or destroyed during normal system operation (i.e. other than at start-up and shut-down of the whole system), what triggers this? For example, a new agent might be created when a user accesses the system for the first time.
- Agent initialization – what needs to be done?
- Agent demise – what clean up needs to be done?

Each agent type should have an agent descriptor containing the just-mentioned information plus the *name* of the agent, a natural language *description* of what this agent does within the system and a list of the *functionalities* from the previous phase that are incorporated within this agent.

In addition, we extract from the functionality descriptors the following information that becomes a part of the agent descriptor:

- What are the goals of this agent?
- What percepts will this agent react to?
- What actions (if any) will it take?
- What data does this agent use or produce?

We also require a brief description of the *interaction protocols* that it uses.

Additional details are also added during *Detailed Design*. The complete descriptor can be seen in Appendix A.

Electronic Bookstore: Case study

For example, consider the following agent descriptor from our electronic bookstore example:

Name: Sales Assistant agent.

Description: Greets customer, follows through site, arranges purchases.

Cardinality: One/customer.

Lifetime: Instantiated on customer arrival at site. Demise when customer logs out or after inactivity period.

Initialization: Obtains cookie. Reads Customer DB, Stock DB.

Demise: Closes open DB connections.

Functionalities included: Online interaction, Purchasing, Welcomer, Book finding.

Uses data: Customer DB, Customer Orders, Books DB, Stock DB.

Produces data: Customer preferences, orders, queries

Goals: Welcome customer; Update customer details; Respond to queries; Facilitate purchases;

Percepts responded to: *Arrival at WWW site, Bank transaction response.*

Actions: *WWW page display, Bank transaction*

Protocols and interactions: *Book finding with Stock Manager, Book ordering and Order status querying with Delivery Manager.*